

移动应用测试教程

Android App Testing Handbook · 基于 ZhujiNote 助记笔记项目

MT2026 移动应用测试 · 大连理工大学软件学院 · 从环境搭建到持续集成的全流程实践

教程概览

配套项目	ZhujiNote 助记笔记 (Jetpack Compose + DeepSeek AI + Clean Architecture)
覆盖课次	L02 环境 · L04 GUI · L05 单元 · L06 Mock · L07 覆盖率 · L08 集成 · L09 CI
技术栈	Gradle 8.9 + AGP 8.5.2 + Kotlin 1.9.24 + Hilt + Room(FTS4) + OkHttp
测试框架	JUnit 4.13.2 · Truth · MockK 1.13.12 · Robolectric 4.13 · Espresso 3.6.1 · Compose UI Test
测试规模	单元 188 (Stage1 117 / Stage2 53 / Stage3 18) + 集成 37 = 225 用例
覆盖率工具	JaCoCo 0.8.12 (三阶段独立 .exec + 累积报告)
累积覆盖率	LINE 65.1% / METHOD 57.9% / BRANCH 53.4%
CI/CD	Jenkins 声明式 Pipeline + GitHub Actions 双轨

7

教程章节

225

测试用例

5

测试框架

100%

通过率

65%

行覆盖率

目录

- 环境搭建与 ADB 入门 L02 · AS / SDK / AVD / adb 命令
- GUI 测试 — Monkey 与 UIAutomator2 L04 · 随机压测 + 结构化自动化
- 单元测试 — JUnit 基础 L05 · JUnit4 + Truth · Stage1
- Mock 测试 — MockK L06 · 隔离依赖 · Stage2
- 测试覆盖率 — JaCoCo + Robolectric L07 · 三阶段覆盖 · Stage3
- 集成测试 — Espresso L08 · Compose UI Test + Hilt
- 持续集成 — Jenkins L09 · Declarative Pipeline

开发环境要素

组件	本项目配置
Android Studio	Panda 4 (D:\Softwares\AndroidStudio)
JDK	AS 自带 JBR 21 (D:\Softwares\AndroidStudio\jbr) , 无需另装
Android SDK	compileSdk 34 / minSdk 24 (%LOCALAPPDATA%\Android\Sdk)
AVD 模拟器	zhuji_avd (Pixel 7, API 34, x86_64, AEHD 加速)
构建工具	Gradle 8.9 + AGP 8.5.2 + Kotlin 1.9.24 + KSP

安装流程

1. 下载 AS → Standard 安装, 自动拉取 SDK 34 + 模拟器镜像
2. 打开项目等 Gradle Sync (已配阿里云镜像加速)
3. Sync 失败点大象图标 Sync Project with Gradle Files

环境变量

```
ANDROID_HOME = %LOCALAPPDATA%\Android\Sdk
PATH += %ANDROID_HOME%\platform-tools
# 验证
adb version
```

镜像加速 (settings.gradle.kts)

```
maven { url =
uri("https://maven.aliyun.com/repository/google") }
maven { url =
uri("https://maven.aliyun.com/repository/public") }
maven { url =
uri("https://maven.aliyun.com/repository/gradle-plugin") }
```

提示 中文路径下 Gradle/Robolectric 可能 ClassNotFoundException, 建议构建目录放英文路径 (如 D:\Projects\zhuji_note) 。

ADB 常用命令

命令	作用
adb devices	列出已连接设备 / 模拟器
adb shell pm list packages	列出所有应用包名
adb install -r app.apk	安装 APK (-r 覆盖)
adb shell am start -n 包名/.Activity	启动指定 Activity
adb shell screencap -p /sdcard/s.png	截屏 (配合 adb pull 取回)
adb shell uiautomator dump	dump 当前界面 XML 层级
adb shell input tap x y / input text "hi"	模拟点击 / 输入 / keyevent 4 返回

一键启动与验证 (本项目脚本)

```
# scripts/start_avd.ps1: 启动+等boot+关动画
.\scripts\start_avd.ps1
```

```
.\gradlew.bat :app:assembleDebug
adb install -r app\build\outputs\apk\debug\app-debug.apk
adb shell am start -n com.zhuji.note/.MainActivity
.\scripts\screenshot.ps1 # → docs/screens/
```

方式	工具	原理
随机压力测试	Monkey（系统自带）	向随机坐标发送随机事件，暴露 Crash / ANR
精确自动化测试	UIAutomator2（Python）	识别界面元素，按控件属性精确操作

Monkey 随机测试

```
adb shell monkey -p com.zhuji.note -v 500
```

参数	说明
-p 包名	限定在某应用内
-s 种子	固定随机序列，可复现
--throttle ms	事件间隔
--pct-touch N	点击事件百分比
-v -v -v	日志级别 L0/L1/L2
--ignore-crashes	崩溃后继续

本项目 scripts/monkey_test.ps1

```
$adb shell monkey -p com.zhuji.note `
--throttle 200 --ignore-crashes `
--ignore-timeouts -v 5000
```

日志含 // CRASH = 崩溃，// NOT RESPONDING = ANR。脚本自动统计数量并写入 docs/reports/monkey-output.txt。

UIAutomator2 (Python)

```
# 安装
pip install uiautomator2 uiautomator2
python -m uiautomator2 init

# 基本脚本
import uiautomator2 as u2
d = u2.connect()
d.app_start("com.zhuji.note")
d.implicitly_wait(10.0)
xml = d.dump_hierarchy()
d.app_stop("com.zhuji.note")
```

定位方式	代码
resourceId	d(resourceId="...:id/fab")
text	d(text="新建笔记")
description	d(description="设置")
坐标	d.click(540, 960)

查看元素：uiauto.dev 启动 Web 查看器；或 adb shell uiautomator dump 后用本项目 scripts/ui_coords.py 解析坐标。

本项目脚本 uiautomator_e2e.ps1 用 adb input 走完整流程并逐页截图；avd_driver.ps1 封装 Find-Node / Tap-Text / Wait-Text / Run-AiAction 等基于 dump XML 的 UI 驱动函数。

维度	Monkey	UIAutomator2
测试目标	稳定性 / 崩溃	功能正确性
理解界面	不需要	需要
编写成本	一行命令	编写脚本
适用阶段	冒烟 / 压力	回归 / 验收

03 单元测试 — JUnit 基础 L05 • Unit Testing

单元测试针对最小单位（方法/类）验证正确性，跑在本机 JVM，不需模拟器，毫秒级。

测试目录

```
app/src/
├ main/      ← 应用代码
├ test/      ← 单元测试 (JVM)
└ androidTest/ ← 集成测试 (设备)
```

依赖

```
testImplementation(libs.junit) // 4.13.2
testImplementation(libs.truth)  // 1.4.4
```

本项目 Stage 1 实例（纯逻辑，零依赖）

```
// WordCountTest 中英混合字数
class WordCountTest {
    @Test fun `chinese counted by char`() {
        assertEquals(wordCount("你好世界"), 4)
    }
    @Test fun `mixed cn en`() {
        assertEquals(wordCount("你好 world"), 3)
    }
}
```

运行 & Stage1 覆盖内容 (117 用例)

```
.\gradlew.bat :app:testDebugUnitTest
# 只跑 Stage1
.\gradlew.bat :app:testDebugUnitTest \
    --tests "com.zhuji.note.stage1.*"
```

JUnit4 注解 & Truth 断言

注解	作用
@Test	标记测试方法
@Before/@After	每个测试前/后
@BeforeClass	类级仅一次
@Ignore	跳过

```
assertThat(r).isEqualTo(expected)
assertThat(list).containsExactly("a", "b").inOrder()
assertThat(n).isWithin(0.01f).of(0.5f)
```

```
// MarkdownToolbarExtendedTest
@Test fun `bold wraps`() {
    val (t, _) = MarkdownToolbar.applyBold("hello", 0, 5)
    assertEquals(t, "***hello**")
}

// DomainModelsTest
@Test fun `note in trash flag`() {
    val n = Note(title="t", content="c", deletedAt=1L)
    assertTrue(n.isInTrash)
}
```

- WordCount 字数
- DomainModels 数据类
- NoteFilter 筛选
- Formatter 时间/预览
- NoteStatistics 统计
- BiLinkParser 双链
- MarkdownToolbar 16项
- TemplateEngine 模板
- Pomodoro 番茄钟
- AiApplyStrategy 合并策略
- AiSerialization 序列化

04 Mock 测试 — MockK L06 • Mocking

被测代码依赖数据库/网络/系统服务时，**Mock** 用虚拟对象替代真实依赖，让测试只关注当前类逻辑，避免外部环境不稳定导致失败。

核心 API

API	作用
<code>mockk<T>()</code>	严格 mock
<code>relaxUnitFun=true</code>	Unit 方法放行
<code>every{...} returns</code>	打桩返回值
<code>coEvery{...}</code>	打桩挂起函数
<code>coJustRun{...}</code>	挂起无返回
<code>coVerify(exactly=N)</code>	验证调用次数
<code>slot<T>()+capture()</code>	捕获参数
<code>mockkObject()</code>	mock 单例

UseCase 测试

```
@Test fun `SaveNote rejects empty`() = runTest {  
    val repo = mockk<NoteRepository>(relaxUnitFun=true)  
    val res = SaveNoteUseCase(repo)(Note("", ""))  
    assertThat(res.isFailure).isTrue()  
}
```

Turbine 测 Flow

```
repo.observeNotes(NoteFilter()).test {  
    assertThat(awaitItem()).isEmpty()  
    cancelAndIgnoreRemainingEvents()  
}
```

Stage 2 覆盖内容 (53 用例)

NoteRepositoryImpl CRUD

ExtendedRepository 排序/统计

RepositoryEdgeCases 边界

UseCases 用例层

DeepSeekClient SSE 流式

DeepSeekClient 401/429

PomodoroState 状态机

WritingGoal 写作目标

Repository 测试 (mock DAO)

```
val noteDao = mockk<NoteDao>(relaxUnitFun=true)  
@Test fun `upsert inserts and refreshes tags`() = runTest {  
    {  
        coEvery { noteDao.insert(any()) } returns 42L  
        coJustRun { noteDao.clearCross(any()) }  
        val repo = NoteRepositoryImpl(noteDao, tagDao)  
        val id = repo.upsert(Note("t", "c", tagIds=listOf(7L, 8L)))  
        assertThat(id).isEqualTo(42L)  
        coVerify(exactly=1) { noteDao.insert(any()) }  
        coVerify(exactly=2) { noteDao.insertCross(any()) }  
    }  
}
```

MockWebServer 测 SSE 流式

```
// DeepSeekClientTest  
server = MockWebServer().also { it.start() }  
client = DeepSeekClient(OkHttpClient(), json,  
    baseUrl = server.url("/").toString().trimEnd('/') )  
  
val body = listOf(  
    ""data: { "choices": [{ "delta": { "content": "你" } } ] } """,  
    ""data: { "choices": [{ "delta": { "content": "好" } } ] } """,  
    "data: [DONE]").joinToString("\n\n")  
server.enqueue(MockResponse().setBody(body)  
    .addHeader("Content-Type", "text/event-stream"))
```

05 测试覆盖率 — JaCoCo + Robolectric L07 • Coverage

覆盖率回答「测了多少代码」：**语句覆盖**（行）、**分支覆盖**（if/when 分支）、**方法覆盖**。

JaCoCo 配置

```
plugins { jacoco }
jacoco { toolVersion = "0.8.12" }
```

三阶段独立采集

```
val stages = listOf("stage1", "stage2", "stage3")
afterEvaluate {
    stages.forEach { /* testStageN → 独立 .exec
                     jacocoStageNReport */ }
    // jacocoCumulativeReport: 合并 3 个 .exec
}
```

覆盖率递进

阶段	工具	新增覆盖范围
Stage 1	纯 JUnit	domain 工具类（格式化/解析/统计）
Stage 2	+ MockK + MockWebServer	Repository + UseCase + AI 客户端业务编排
Stage 3	+ Robolectric	Room 真实 SQL + DataStore + Theme/Color 解析

Robolectric — JVM 上模拟 Android Runtime

纯 JUnit 无法访问 Android Framework（Room `_Impl`、Color、资源）。Robolectric 在 JVM 模拟运行时，无需模拟器即可测这些代码。

```
testImplementation(libs.robolectric) // 4.13
```

运行

```
.\gradlew.bat :app:jacocoStage1Report `
:app:jacocoStage2Report :app:jacocoStage3Report
.\gradlew.bat :app:jacocoCumulativeReport
# 报告 → app/build/reports/coverage/
#      cumulative/html/index.html
```

排除规则

```
val excludes = listOf(
    "**/R.class", "**/*Hilt_*.*", "**/di/**",
    "**/ui/screens/**", "**/ui/theme/**")
// UI 层由 Espresso 集成测试覆盖
```

```
@RunWith(RobolectricTestRunner::class)
@Config(sdk = [33])
class RoomDatabaseTest {
    @Before fun create() {
        db = Room.inMemoryDatabaseBuilder(ctx,
            ZhujiDatabase::class.java)
            .allowMainThreadQueries().build()
    }
    @Test fun `insert and observe`() = runTest {
        val id = db.noteDao().insert(NoteEntity(...))
        assertThat(db.noteDao().observeAll().first()
            .first().id).isEqualTo(id)
    }
}
```

Stage3 (18 用例) RoomDatabaseTest（CRUD/软删/搜索/聚合）· UserPreferencesDataStoreTest（主题/Key/字号）· ThemeAndComposeTest（亮暗 ColorScheme + Typography）。Robolectric 镜像可设 `robolectric.dependency.repo.url` 指向阿里云加速。

集成测试验证多模块协作，跑在**真实设备/模拟器**，测 UI 交互、Activity 导航、组件间数据传递。

维度	单元测试	集成测试
运行环境	JVM	设备 / 模拟器
目录	src/test/	src/androidTest/
依赖声明	testImplementation	androidTestImplementation
运行命令	gradlew test	gradlew connectedAndroidTest
验证重点	逻辑正确性	UI 交互 + 组件协作

依赖 + Hilt Runner

```
androidTestImplementation(libs.espresso.core)
androidTestImplementation(libs.espresso.intents)
androidTestImplementation(libs.androidx.compose.ui.test.junit4)
androidTestImplementation(libs.hilt.android.testing)

// build.gradle.kts
testInstrumentationRunner =
    "com.zhuji.note.HiltTestRunner"
```

核心 API

API	作用
onNodeWithText("...")	按文本找
onNodeWithContentDescription	按描述找
.performClick()	点击
.performTextInput("...")	输入
.assertExists()	断言存在

集成测试清单 (37 用例) & 运行

测试类	用例
EditorE2ETest	10
NavigationE2ETest	10
NoteFlowE2ETest	10
CommonComposeTest	3
ThemeRendersTest	2
MainActivityIntegrationTest	2

本项目 EditorE2ETest

```
@HiltAndroidTest
@RunWith(AndroidJUnit4::class)
class EditorE2ETest {
    @get:Rule(order=0) val hilt = HiltAndroidRule(this)
    @get:Rule(order=1) val rule =
        createAndroidComposeRule<MainActivity>()

    @Before fun setup() {
        hilt.inject()
        rule.awaitContentDescription("新笔记")
    }
    @Test fun editorTitleInput() {
        rule.onNodeWithContentDescription("新笔记").performClick()
        rule.awaitText("标题")
        rule.onNode(hasText("标题")).performTextInput("My Title")
        rule.onNodeWithText("My Title").assertExists()
    }
}
```

```
.\scripts\start_avd.ps1
.\gradlew.bat :app:connectedDebugAndroidTest
# 报告 → app/build/reports/
#         androidTests/connected/index.html
```

防 flaky animationsDisabled=true + start_avd 关闭三项动画 scale; 用 awaitText/awaitContentDescription 替代硬 sleep; ActivityScenarioRule 自动隔离状态。

CI: 频繁提交, 每次自动构建+测试, 尽早发现集成问题。四要素: 版本控制 · 自动化构建 · 自动化测试 · 报告通知。

安装 & 初始化

```
java -jar jenkins.war --httpPort=8080
# 1. 取 initialAdminPassword 解锁
# 2. 安装推荐插件(git/gradle/pipeline)
# 3. 创建管理员 → 配置 URL
```

Android 工具配置

- JDK → JBR 21
- Gradle → 用项目自带 Wrapper
- 环境变量 **ANDROID_HOME** → SDK

启动 (支持本地仓库)

```
java -Dhudson.plugins.git.GitSCM.ALLOW_LOCAL_CHECKOUT=true -jar jenkins.war --httpPort=8080
```

本项目 Jenkinsfile (声明式)

```
pipeline {
    agent any
    environment {
        ANDROID_HOME = '...\Android\Sdk'
        JAVA_HOME     = '...\AndroidStudio\jbr'
    }
    stages {
        stage('Checkout') { steps { checkout scm } }
        stage('Setup') { steps {
            bat 'echo sdk.dir=... > local.properties' } }
        stage('Build APK') { steps {
            bat '...\gradlew.bat :app:assembleDebug' } }
        stage('Tests + JaCoCo') {
            steps { bat '...\gradlew.bat ...jacocoCumulativeReport' }
        }

        post { always {
            junit 'app/build/test-results/**/*.xml'
            publishHTML(/* JaCoCo cumulative */) } }
    }
    stage('Archive APK') { steps {
        archiveArtifacts '.../debug/*.apk' } }
    }
}
```

Pipeline 各阶段

Stage	动作
Checkout	拉取源码
Setup	生成 local.properties 指向 SDK
Build APK	Gradle 编译 debug APK
Tests + JaCoCo	三阶段单测 + 累积覆盖率, 归档 JUnit/HTML 报告
Archive APK	归档 APK 产物

命令行构建 (不依赖 Jenkins)

```
.\gradlew.bat :app:assembleDebug           # 编 APK
.\gradlew.bat :app:testDebugUnitTest        # 188 单测
.\gradlew.bat :app:jacocoCumulativeReport    # 覆盖率
.\gradlew.bat :app:connectedDebugAndroidTest # 37 集成(需 AVD)
```

Git Push → Jenkins 检出 → Gradle Build → 单元测试 → JaCoCo 报告 → 归档 APK